

P1302R0

Implicit Module Partition Lookup

Published Proposal, 2018-10-07

This version:

<https://wg21.link/p1302r0>

Authors:

[Isabella Muerte](#)

[Richard Smith](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1302](#)

Current Source:

slurps-mad-rips/papers/proposals/implicit-module-partition.bs

Abstract

While we cannot enforce a project or directory layout for module interface units, we can encourage a general convention that developers, build systems, and compiler vendors can rely on for "quick paths" when developing software in C++

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design

4 Examples

4.1 Hierarchical

4.2 Splayed

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

As the advent of modules approaches, build systems and developers are still waiting on the final bits that will make it into C++20. However, because of various limitations, the standard cannot *enforce* specific convention, merely encourage them. This paper seeks to provide a possible convention that reduces work required by build systems, general effort from compilers (e.g., they will not need to implement a local socket based server for information passing), and to make the lives of developers easier as they will get to experience a fairly consistent development process when moving between or across projects.

§ 3. Design

The design for this convention is as follows:

A so-called *root module* is placed inside of a directory. This module has the name `module.{ext}` (where `{ext}` is some developer decided file extension. However, this paper recommends `.cxx` as that is the environment variable used by plenty of C++ build systems to represent the C++ compiler, whereas the environment variable `CPP` represents the **C PreProcessor**) and it holds the primary set of imports and exports for a given module. Module interface units in the same directory are treated as "implicit" partitions. So `import :list`, means the compiler will look for a file named `list.{ext}`. This ensures that all files have the same file extension in a given module directory. This is simply to construct the BMI, and does not automatically extend to compiling each file as an object file. This is still left up to build systems so that existing distributed build system workflows are not interrupted (such as in the case of tools like Icecream, distcc, or sccache).

Where things get interesting is when a user desires to import *another module* into a root module. Given the following directory layout:

```
├── src
│   └── core
│       ├── module.cxx
│       ├── list.cxx
│       └── io
│           ├── module.cxx
│           └── file.cxx
```

We can assume that, perhaps, the source for `core/module.cxx` looks something like:

```
export module core;

export import core.io;
import :list;
```

In other languages, this would imply that the compiler has to now recurse into the `core/io` directory. We do not do this. Instead, the build system is required to have seen the `export import` and passed that directory along to the compiler first. This is a combination of the various modules systems, but has the following properties:

1. It does not make the compiler a build system
2. Existing work that has been done to handle dependency management does not need to be thrown away.
3. The `core.io` module *does not* have to exist in the directory `core/io`. Rather, it can exist technically anywhere
4. Build systems can have a guaranteed fallback location if developers don't want to have to manually specify the location of each and every module.
5. This doesn't actually tie the compiler to a filesystem approach, as this is just a general convention.
6. Build systems are free to implement, additional conventions, such as the Pitchfork Layout and enforce it for modules having legacy non-module code in the same project layout.
7. It allows developers to view modules as hierarchical, even if they aren't. This means that, if treating modules as a hierarchy becomes widespread enough, the standard could possibly enforce

modules as hierarchies in the future.

8. Platforms where launching processes are expensive can take advantage of improved throughput when reading from files.
9. Build systems and compilers are free to take an optimization where only the modified times of a directory are checked before the contents of each directory are checked. On every operating system (yes, every operating system), directories change their timestamp if any of the *files* contained within change, but do not update if child directories do as well. While some operating systems permit mounting drives and locations without modified times, doing so breaks nearly every other build system in existence. Thus we can safely assume that a build system does not need to reparse or rebuild a module if its containing directory has not changed.

§ 4. Examples

The following two examples show how implicit module partition lookup can be used for both hierarchical and "splayed" directory layouts.

§ 4.1. Hierarchical

This sample borrows from the above example. Effectively, to import `core.io`, one must build it before building `core` simply because the build system assumes that `core.io` refers to a directory named `core/io` from the project root.

```
├── src
│   ├── core
│   │   ├── module.cxx
│   │   ├── list.cxx
│   │   └── io
│   │       ├── module.cxx
│   │       └── file.cxx
```

§ 4.2. Splayed

This approach is one that might be more commonly seen as C++ developers move from headers to modules.

```
├── core
│   ├── module.cxx
│   └── list.cxx
└── io
    ├── module.cxx
    └── file.cxx
```

In the above layout, `core.io` is located in `./io`, rather than under the `./core` directory. A sufficiently simple build system could be told that `core.io` resides under `./io` and not to rely on some kind of hierarchical directory layout.